

"Não existe nuvem, você só está usando a máquina de outra pessoa!"

Deploy e Projeto Final

Paulo Lisboa de Almeida

Objetivo

O objetivo da aula é carregar uma API em um servidor remoto.

O código fonte ficará disponível publicamente em seu GitHub.

Fará parte de seu portfólio.

Quanto mais complexo e interessante for a sua API, melhor para seu portfólio.

Valerá como a nota da aula (não haverá Quiz).

Atenção

Siga os passos com **cuidado**.

Ao final da aula, **remova completamente** a máquina da nuvem para evitar:

- Consumo de créditos.
- Problemas de segurança.

Você deixará disponível apenas o código fonte e talvez um docker no GitHub.

- Se alguém quiser usar o seu serviço, basta clonar o repositório em uma máquina.

Repositório

Crie um repositório no GitHub com o nome `genaiufpr-fastapi`.

Deixe público.

Clone para sua máquina local.

Ambiente

Crie um ambiente conda:

```
conda create --name clima_api
```

Ative o ambiente

```
conda activate clima_api
```

Instale

Instale os pacotes necessários no ambiente conda.

Vamos começar com:

```
conda install fastapi
```

```
conda install requests
```

Projeto

Vamos começar com uma API Simples que retorna a temperatura atual de uma cidade.

API

```
import requests
from fastapi import FastAPI

GEO_URL = "https://geocoding-api.open-meteo.com/v1/search"
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"
app = FastAPI()

@app.get("/temperatura-cidade")
def temperatura_cidade(nome_cidade: str):
    geo_params = {
        "name": nome_cidade,
        "count": 1
    }
    geo_response = requests.get(GEO_URL, params=geo_params)
    geo_data = geo_response.json()
    lat = geo_data["results"][0]["latitude"]
    lon = geo_data["results"][0]["longitude"]

    weather_params = {
        "latitude": lat,
        "longitude": lon,
        "current_weather": True
    }
    weather_response = requests.get(WEATHER_URL, params=weather_params)
    weather_data = weather_response.json()

    return weather_data["current_weather"]["temperature"]
```


API

```
import requests
from fastapi import FastAPI

GEO_URL = "https://geocoding-api.open-meteo.com/v1/search"
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"
app = FastAPI()

@app.get("/temperatura-cidade")
def temperatura_cidade(nome_cidade: str):
    geo_params = {
        "name": nome_cidade,
        "count": 1
    }
    geo_response = requests.get(GEO_URL, params=geo_params)
    geo_data = geo_response.json()
    lat = geo_data["results"][0]["latitude"]
    lon = geo_data["results"][0]["longitude"]

    weather_params = {
        "latitude": lat,
        "longitude": lon,
        "current_weather": True
    }
    weather_response = requests.get(WEATHER_URL, params=weather_params)
    weather_data = weather_response.json()

    return weather_data["current_weather"]["temperature"]
```

Nossa API ainda chama outras para resolver o problema!

Push

Envie para o Git.

```
git add --all
```

```
git commit -m "Criado server básico"
```

```
git push origin main
```

Docker



Facilitar o deploy de aplicações.

Separar aplicação de infraestrutura.

Uso de containers.

- Ambiente virtual.

- Devem conter tudo que a aplicação precisa para executar.

<https://docs.docker.com/get-started/docker-overview>

Instalação

Verifique se você tem o docker instalado através de:

```
docker --version
```

Caso você não tenha o Docker instalado na sua máquina, siga os passos descritos na documentação oficial para instalar.

<https://docs.docker.com/engine/install/ubuntu>

Exportando Libs Conda

Exporte as dependências do Conda através do comando:

```
conda env export --from-history > environment.yml
```

Será criado um arquivo chamado `environment.yml` no seu projeto.

Exportando Libs Conda

Exporte as dependências do Conda através do comando:

```
conda env export --from-history > environment.yml
```



Usar somente o que foi explicitamente instalado. Remova essa opção e compare.

environment.yml

O arquivo deve ficar parecido com esse:

```
name: clima_api
```

```
channels:
```

```
- defaults
```

```
dependencies:
```

```
- python=3.14
```

```
- fastapi
```

```
- requests
```

```
- uvicorn
```

```
prefix: /home/.../clima_api
```

Adicione. Para saber qual versão usar,
execute `python --version`

Adicione

Remova essa linha.

Dockerfile

Crie um arquivo chamado Dockerfile, e insira:

```
FROM continuumio/miniconda3
```

```
WORKDIR /app
```

```
COPY environment.yml .
```

```
RUN conda env create -f environment.yml
```

```
COPY . .
```

```
CMD ["conda", "run", "--no-capture-output", "-n", "clima_api", "uvicorn", "clima_api:app",  
"--host", "0.0.0.0", "--port", "8000"]
```


Deploy

Para realizar o deploy, realize os seguintes passos.

Detalhe: dependendo de como o docker foi instalado na sua máquina, alguns passos podem exigir `sudo`.

```
docker build -t clima-api .  
docker images  
docker run -p 8000:8000 clima-api
```

`docker images` serve apenas para listar as imagens disponíveis e verificar o resultado do build.

Commit

Se você ainda não o fez, agora é uma boa hora para realizar o commit e o push.

Oracle Cloud

Suba uma instância simples no Oracle Cloud.

Instância do **tier gratuito**.

Escolha Ubuntu como sistema operacional.

Conecte via `ssh` na instância.

Carregando no servidor

Temos duas opções básicas para carregar o docker no servidor Oracle Cloud.

1. Exportar a imagem pronta diretamente para o servidor.
 - a. **Vamos fazer.**
 - b. Comum em ambientes corporativos.
 - c. Mais simples, mas envolve transferir uma grande massa de dados para o servidor.
2. Clonar o projeto Git no servidor, e fazer o build a imagem diretamente no servidor.
 - a. Espera-se que quem clonar seu projeto no GitHub faça isso.
 - b. Evita manter builds que ocupam muito espaço no GitHub (não faça commit da imagem) e transferir grandes massas de dados.
 - c. Exige que a API completa do Docker e do Git estejam disponíveis no servidor.
 - d. Para fazer dessa forma, basta reproduzir alguns comandos que fizemos localmente, como o `docker build`, diretamente no servidor.

Carregando no servidor

Exportar a imagem docker como um .tar

```
docker save clima-api > clima-api.tar
```

Depois disso, copie `clima-api.tar` para sua máquina na Oracle Cloud.

Docker no Servidor

Instale o Docker no servidor:

```
sudo apt install docker-ce docker-ce-cli containerd.io  
docker-buildx-plugin docker-compose-plugin
```

Carregue a imagem docker enviara para o Docker.

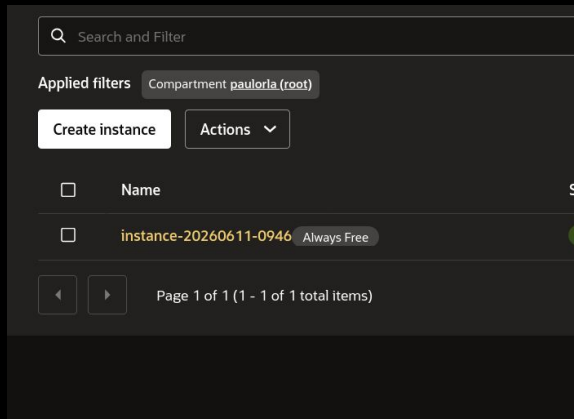
```
docker load < clima-api.tar
```

Execute a API na porta 8000.

```
docker run -p 8000:8000 clima-api
```

Abrindo a porta

Clique na instância em execução:



Abrindo a Porta

Em Networking, clique na subrede (subnet).

Details

Networking

Storage

Security

Management

OS Management

Monitoring

Work requests

Tags

Primary VNIC

Public IPv4 address

147.15.30.173

Private IPv4 address

10.0.0.238

Route table

Default Route Table for vcn-20260611-0947

Network security groups

—

This instance's traffic is controlled by its firewall rules in addition to the associated [subnet's](#) security lists and the VNIC's network security groups.

Edit

Subnet

subnet-20260611-0947

Private DNS record

Enable

Hostname

instance-20260611-0946

Internal FQDN

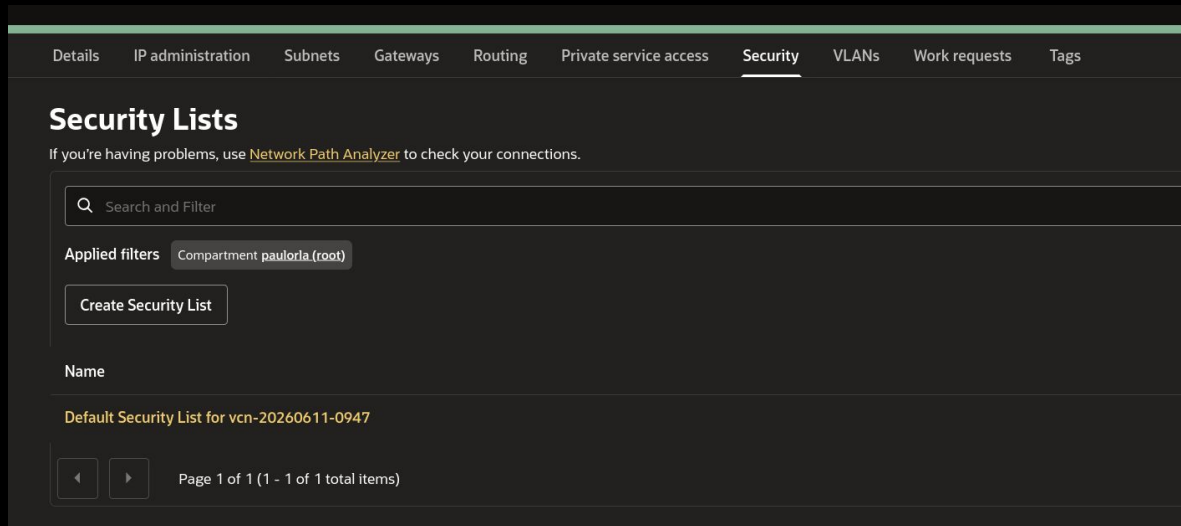
instance-20260611-0946.subnet06110949.vcn06110949.oraclevcn.com

Copy

Attached VNICs

Abrindo a Porta

Clique em Security, e abra a lista de segurança padrão (Default Security ...).



The screenshot shows the AWS IAM console interface. At the top, there is a navigation bar with tabs: Details, IP administration, Subnets, Gateways, Routing, Private service access, **Security** (which is selected and underlined), VLANs, Work requests, and Tags. Below the navigation bar, the main heading is "Security Lists". A sub-header text reads: "If you're having problems, use [Network Path Analyzer](#) to check your connections." Below this is a search bar with a magnifying glass icon and the text "Search and Filter". Under the search bar, it says "Applied filters" followed by a button labeled "Compartment pauloria (root)". There is a "Create Security List" button. Below the button, the section is titled "Name". A single item is listed: "Default Security List for vcn-20260611-0947". At the bottom, there are navigation controls: a left arrow, a right arrow, and the text "Page 1 of 1 (1 - 1 of 1 total items)".

Details IP administration Subnets Gateways Routing Private service access **Security** VLANs Work requests Tags

Security Lists

If you're having problems, use [Network Path Analyzer](#) to check your connections.

Search and Filter

Applied filters **Compartment pauloria (root)**

Create Security List

Name

Default Security List for vcn-20260611-0947

Page 1 of 1 (1 - 1 of 1 total items)

Em Security Rules, em regras de entrada (ingress), adicione como está na imagem.

The screenshot displays the AWS IAM console interface for editing a security rule. The left sidebar shows the navigation menu with 'Security rules' selected. The main content area is titled 'Edit Ingress Rule' and shows the configuration for 'Ingress Rule 1'. The rule is currently 'Stateless' and allows TCP traffic for ports: all. The configuration fields are as follows:

- Source Type:** CIDR
- Source CIDR:** 0.0.0.0/0
- IP Protocol:** TCP
- Source Port Range:** (empty)
- Destination Port Range:** 8000
- Description:** (empty)

At the bottom right of the configuration area, there is a button labeled '+ Another Ingress Rule'.

Default Security List for
Security List
Instance traffic is controlled by firewall rules on ec2 instances.

Details **Security rules** Tags

Ingress Rules

Search and Filter

Add Ingress Rules Actions

	Stateless	Source
☐	No	0.0.0.0/0
☐	No	0.0.0.0/0
☐	No	10.0.0.0/24
☐	No	0.0.0.0/0

Page 1 of 1 (1 - 4 of 4 total items)

Egress Rules

Search and Filter

Add Egress Rules Actions

Faça você mesmo

Crie você mesmo uma API de sua escolha, para manter o código fonte no GitHub, e também para subir no Oracle Cloud para testes. Não esqueça de **apagar a máquina da Oracle Cloud** para evitar problemas depois de testar o projeto.

Faça qualquer projeto de API que lhe agradar. Caso esteja sem ideias, algumas sugestões:

1. Sofisticar a API de clima. Criar serviços que retornam, por exemplo, imagem com um gráfico indicando o clima nas últimas 24 horas na cidade passada como parâmetro.
2. O mesmo que 1, mas agora com cotações de moedas ou de ações da bolsa de valores.
3. Carregar um modelo de machine learning simples já treinado na máquina (exemplo: um modelo que identifica gatos e cães em imagens). Criar uma API que usa esse modelo, onde ela recebe uma imagem, e a classifica usando o modelo.
 - a. Fica mais interessante se você subir duas máquinas no Oracle Cloud. Uma que só contém o modelo, e não expõe nenhuma porta ou serviço para o mundo, e outra que tem a API em si, que conversa com a máquina do modelo para obter a resposta.